

AULA 2 · ETAPA 1 · FUNDAMENTOS

Do zero ao primeiro nó

Comunicação em ROS 2 no seu repositório



Prof. Dácio Moreira de Souza · terça 28/07 · 07h00–09h30 · SJ205

Aula 1 → hoje



Repositório criado

cada um tem o seu projeto-pb-usuario no GitHub



Ambiente em pé

WSL2 + ROS 2 (ou terminando — sem problema)



Branches

dev + entrega-* via init-branches.sh

Quem não fechou o aceite/setup: senta na faixa de apoio — resolvemos hoje.

CHECKLIST DE ENTRADA

Você já devia ter concluído (senão, começamos por aqui)

- 1 Conta GitHub + assignment aceito + convite do e-mail *manual-do-aluno-github*
- 2 WSL2 + Ubuntu 22.04 (jammy) *setup • passos 1-2*
- 3 ROS 2 Humble — ros2 doctor verde *setup • passos 3-5*
- 4 git + gh (login) + uv *setup • passo 6*
- 5 Repo clonado no WSL + init-branches.sh *README / manual*

Confira o ambiente em 5 segundos:

```
curl -sSL https://raw.githubusercontent.com/Prof-Dacio-INFNET/PBRoboticos_prof_dacio/main/recursos/check-ambiente.sh | bash
```

Faltou algo? Sem pânico — a primeira parte da aula é para terminar. Guias no material e no site.

HOJE

O que você sai sabendo

1 Ambiente e repositório funcionando
fechar as pendências da Aula 1

2 Escrever e rodar seus primeiros nós
publisher + subscriber, de verdade

3 Entender pub/sub e serviços
na prática, com foco no TP1

4 Projeto escolhido
e PROJETO.md iniciado

RECAP

Tudo é um grafo de nós que trocam mensagens



Nó = programa que faz uma coisa • Tópico = o canal por onde a mensagem viaja • a mensagem tem um TIPO.

Hoje vamos criar esses nós — no SEU repositório.

Tópicos = publish / subscribe



Correio contínuo: um nó PUBLICA, outros ASSINAM — sem saber quem está do outro lado.

Publisher

cria e envia mensagens no tópico

```
create_publisher(msg, '/tópico', 10)
```

Subscriber

recebe via callback

```
create_subscription(msg, '/tópico', cb, 10)
```

Tipos comuns: std_msgs/String, sensor_msgs/Image (a câmera do TP1).

Serviços = pergunta / resposta



Um cliente CHAMA, um servidor RESPONDE uma vez. Síncrono, pontual.

Exemplo do TP1: /vision/status

```
cliente: "quantos objetos no último frame?" → servidor: "3"
```

Hoje vamos rodar um serviço /contagem que responde quantas mensagens já passaram — o irmão mais novo do /vision/status.

Tópico × Serviço

Tópico (pub/sub)

- fluxo contínuo / streaming
- muitos ouvintes
- sem resposta
- ex.: imagem da câmera, /cmd_vel

Serviço (req/resp)

- consulta pontual
- 1 pergunta → 1 resposta
- síncrono
- ex.: quantos objetos? liga/desliga

QoS: por que às vezes “não chega”



Publisher e subscriber precisam de configurações de qualidade (QoS) compatíveis. Se divergem muito, a mensagem não conecta.

Hoje: não se preocupe — os padrões funcionam. Guarde só o sintoma: “publiquei, mas o subscriber não recebe” pode ser QoS (ou tópico/nome errado, ou faltou source).

POR QUE ISSO IMPORTA

Isto É o esqueleto do seu TP1

publicador → /camera/status

→ vira o publisher da câmera → /camera/image_raw

assinante

→ vira o subscriber que processa a imagem (HSV, faces)

serviço /contagem

→ vira o /vision/status (nº de objetos)

Adiantar hoje = começar o TP1 com o pé direito (leitura oficial do TP1: 11/08).

AO VIVO

Os 3 nós conversando



Acompanhem os nomes dos nós e tópicos aparecendo.

- ▶ `ros2 launch aula02_comunicacao comunicacao.launch.py`
- ▶ `ros2 topic echo /camera/status`
- ▶ `ros2 service call /contagem std_srvs/srv/Trigger "{}"`
- ▶ `rqt_graph`

Duas faixas, ninguém parado

Faixa A — ROS 2 ok

- Copie o exemplo p/ `ros2_ws/src`
- `colcon build --symlink-install + source`
- Rode o launch e o `rqt_graph`
- Faça o exercício guiado (próximo slide)

Faixa B — ainda instalando

- Foco: terminar o setup
- Meta: `ros2 doctor` verde + repo clonado + `init-branches`
- Acompanha o hands-on pela projeção
- Recebe ajuda do professor/monitores de fato

O esqueleto do seu TP1, passo a passo

- 1 `git checkout dev` · copie `exemplos/pacote_minimo` para `ros2_ws/src/`
- 2 Renomeie o pacote para algo do seu projeto (ex.: `percepcao_meuprojeto`)
- 3 No publisher, publique um “status” do seu domínio (texto ou contador)
- 4 Adicione um SEGUNDO subscriber que reaja à mensagem
- 5 Ponte-TP1: troque o tópico para `/camera/status` · rode `rqt_graph`
- 6 `git commit -m "Primeiros nós de comunicação" && git push` (na dev)

Rápido demais? Adicione um serviço que conta os objetos (base do `/vision/status`).

Comandos que você mais vai usar hoje

Compilar/rodar

```
colcon build --symlink-install  
source install/setup.bash  
ros2 launch PKG comunicacao.launch.py
```

Inspecionar

```
ros2 topic list / echo /camera/status  
ros2 service call /contagem std_srvs/srv/Trigger  
"{}"  
rqt_graph
```

Erro nº 1 do dia: esquecer o source install/setup.bash depois do build.

Seu projeto: escolha e PROJETO.md



Rodada rápida: cada um diz o projeto escolhido; eu aprovo ou ajusto na hora.

- Objetivo e aplicação prática
- O que percebe (câmera + classes) e onde navega
- Sensores/hardware (real ou simulado)
- Plano TP a TP + riscos

Comece o PROJETO.md do seu repositório hoje — o catálogo de projetos está no material.

Fechar pendências de aceite e setup



Aceitou o assignment E aceitou o convite no e-mail (senão dá 404)



Clonou DENTRO do WSL (~), não em /mnt/c



Rodou ./scripts/init-branches.sh (tem dev + entrega-*)



ros2 doctor verde (ou plano para terminar em casa)

ATÉ A PRÓXIMA TERÇA

Tarefas da semana

1

Terminar o setup

ros2 doctor verde; repositório clonado e com branches

2

Concluir o exercício dos nós

publisher + subscriber; push na dev

3

Rascunhar o PROJETO.md

proposta do seu projeto (use o catálogo)

4

Desafio 1 (opcional) + ler o manual do aluno

treino p/ o TP1; fluxo de branches/entrega

Próxima aula (04/08): interfaces e mensagens — a caminho do TP1. Dúvidas: Infnet.Online.